

FUNÇÕES VIRTUAIS, CLASSES ABSTRATAS E INTERFACES

Prof. André Backes

Polimorfismo

- Capacidade de um mesmo nome manifestar comportamentos diferentes conforme o contexto
 - Do grego *muItas formas*
- Pilar que permite extensibilidade e reuso elegante de código
 - Permite o que um código escrito em termos de uma interface ou de um tipo genérico (template) funcione corretamente com múltiplas implementações concretas

Polimorfismo

- Duas grandes categorias de polimorfismo
 - Estático ou de tempo de compilação
 - Dinâmico ou de tempo de execução

Polimorfismo Estático

- Engloba sobrecarga de funções e operadores e, de maneira mais poderosa, *templates*
 - O compilador escolhe em tempo de compilação qual versão chamar
 - Permite otimizações mais agressivas e elimina *overhead em runtime*
 - Resulta em código genérico e otimizado

Polimorfismo Dinâmico

- Implementado por meio de funções virtuais
 - Você programa “para a interface” (classe base) e permite que cada classe derivada forneça sua própria implementação (método sobrescrito)
 - O programa escolhe em tempo de execução a versão correta do método
 - Faz uso de um ponteiro ou referência para a classe base

Polimorfismo Dinâmico

- Funcionamento
 - Método da classe é marcado como virtual
 - Compilador organiza uma tabela interna de despacho (*vtable*)
 - Armazena os endereços das implementações vigentes
 - O objeto da classe com virtuais possui um ponteiro oculto (*vptr*)
 - Aponta para essa tabela virtual
 - Isso permite escolher, em tempo de execução, a implementação apropriada com base no tipo real do objeto referenciado

Polimorfismo Dinâmico

- Permite trocar implementações sem alterar o código cliente
 - Mecanismo essencial para programação orientada a objetos extensível
 - Isso facilita a escrita de *frameworks*, sistemas de *plugins*, hierarquias de componentes gráficos e arquiteturas orientadas a interfaces

Ponteiro e referência para classe

- Não é possível atribuir o endereço de uma variável a um ponteiro de outro tipo
 - Tipos distintos podem ter representações, alinhamentos e tamanhos e operações diferentes
 - Erro de compilação

```
int main() {  
    int *p;  
    float y = 20.0;  
  
    //Erro de compilação  
    p = &y;  
    int &r = y;  
  
    return 0;  
}
```


Ponteiro e referência para classe

- Polimorfismo dinâmico depende ponteiro/referência para a classe base
 - C++ garante semanticamente que um objeto da classe derivada contém um subobjeto correspondente à sua classe base
 - Logo, podemos armazenar o endereço de um objeto de uma classe derivada em um ponteiro para sua classe base

```
class Pessoa {
protected:
    string nome;
public:
    Pessoa(string _nome): nome(_nome){}

    void exibirNome() {
        cout << "Nome: " << nome << endl;
    }
    void exibirCargo() {
        cout << "Cargo: ??????" << endl;
    }
};

class Empregado : public Pessoa {
private:
    string cargo;
public:
    Empregado(string _nome, string _cargo)
        : Pessoa(_nome), cargo(_cargo) { }

    void exibirCargo() {
        cout << "Cargo: " << cargo << endl;
    }
};
```

Ponteiro e referência para classe

- Problema
 - Empregado reimplementa a função **exibirCargo()**
 - Acesso por ponteiro/referência não consegue exibir os dados corretamente
 - Ponteiro/referência só pode acessar a parte do objeto que corresponde à classe base, Pessoa

```
int main() {  
    Empregado em("Eduardo", "Professor");  
    em.exibirNome();  
    em.exibirCargo();  
    // Acesso via ponteiro  
    Pessoa *pp = &em;  
    pp->exibirNome();  
    pp->exibirCargo();  
    // Acesso via referência  
    Pessoa &rp = em;  
    rp.exibirNome();  
    rp.exibirCargo();  
  
    return 0;  
}
```

Saída:

Nome: Eduardo
Cargo: Professor

Nome: Eduardo
Cargo: ??????

Nome: Eduardo
Cargo: ??????

Ponteiro e referência para classe

- Por que isso é um problema?
 - Imagine ter que criar uma sobrecarga de **exibirDados()** para a classe base e cada classe derivada
 - Isso é um problema quando se tem muitas derivadas
 - Poderíamos usar um *template*, mas isso traz consigo alguns problemas
- Solução: criar uma função virtual

```
void exibirDados(Pessoa &rp) {  
    rp.exibirNome();  
    rp.exibirCargo();  
}  
  
void exibirDados(Empregado &re) {  
    re.exibirNome();  
    re.exibirCargo();  
}  
  
int main() {  
    Pessoa pe("Andre");  
    Empregado em("Eduardo", "Professor");  
  
    exibePessoa(pe);  
    exibeEmpregado(em);  
  
    return 0;  
}
```

Função Virtual

- Método declarado usando **virtual**
 - Indica ao compilador que ele pode ser sobrescrito por uma classe derivada
 - Decisão de qual implementação usar é feita na execução do programa (*late binding*) e não mais em tempo de compilação (*early binding*)
 - Faz uso da tabela interna de despacho
 - *vtable* - *virtual table*

```
class Pessoa {
protected:
    string nome;
public:
    Pessoa(string _nome): nome(_nome) {}

    void exibirNome() {
        cout << "Nome: " << nome << endl;
    }
    virtual void exibirCargo() {
        cout << "Cargo: ??????" << endl;
    }
};

class Empregado : public Pessoa {
private:
    string cargo;
public:
    Empregado(string _nome, string _cargo)
        : Pessoa(_nome), cargo(_cargo) {}

    virtual void exibirCargo() {
        cout << "Cargo: " << cargo << endl;
    }
};
```

Função Virtual

- O programa agora consegue chamar a versão mais específica da hierarquia para um método que foi reimplementado
 - Empregado reimplementa a função **exibirCargo()**
 - Acesso por ponteiro/referência exibe os dados corretamente

```
int main() {  
    Empregado em("Eduardo", "Professor");  
    em.exibirNome();  
    em.exibirCargo();  
    // Acesso via ponteiro  
    Pessoa *pp = &em;  
    pp->exibirNome();  
    pp->exibirCargo();  
    // Acesso via referência  
    Pessoa &rp = em;  
    rp.exibirNome();  
    rp.exibirCargo();  
  
    return 0;  
}
```

Saída:

Nome: Eduardo
Cargo: Professor

Nome: Eduardo
Cargo: Professor

Nome: Eduardo
Cargo: Professor

Função Virtual

- Permite tratar toda a hierarquia de classes de forma genérica
 - Várias classes filhas ou derivadas
- Função **imprimeObjeto()**
 - Capaz de manipular a impressão dos dados de todas as classes derivadas

```
class Ponto{
public:
    virtual void exibeNome() { cout << "Ponto\n"; }
};

class Reta: public Ponto{
public:
    virtual void exibeNome() { cout << "Reta\n"; }
};

class Quadrado: public Reta{
public:
    virtual void exibeNome() { cout << "Quadrado\n"; }
};

class Cubo: public Quadrado{
public:
    virtual void exibeNome() { cout << "Cubo\n"; }
};

void imprimeObjeto(Ponto &p){
    cout << "Objeto: ";
    p.exibeNome();
}
```

Função Virtual

- Função **imprimeObjeto()**
 - Não é mais necessário reimplementar essa função para cada tipo de objeto
 - Acesso a implementação correta da função com base no ponteiro/referência da classe base

```
int main() {  
    Reta r;  
    Cubo c;  
    Quadrado q;  
  
    imprimeObjeto(r);  
    imprimeObjeto(c);  
    imprimeObjeto(q);  
  
    return 0;  
}
```

Função Virtual

- Métodos que não foram definidos como virtuais não são polimórficos
 - Se o método não for virtual, a versão chamada é sempre a da classe base
 - **exibeValor()** é escolhido em tempo de compilação

```
class Base {
public:
    virtual void exibeNome() { cout << "Base\n"; }
    void exibeValor(int x) {
        cout << "Base: " << x << "\n";
    }
};

class Derivada : public Base {
public:
    virtual void exibeNome() { cout << "Derivada\n"; }
    void exibeValor(int x) {
        cout << "Derivada: " << x << "\n";
    }
};

int main() {
    Derivada d;
    Base &b = d;

    b.exibeNome();
    b.exibeValor(5);

    return 0;
}
```

Saída:

Derivada
Base: 5

Função Virtual | Desvantagens

- Sobrecarga em tempo de execução
 - Acesso a tabela virtual, causando perda de desempenho
- Otimização em tempo de compilação fica prejudicada
 - O compilador nem sempre pode aplicar algumas técnicas de otimização
- Exige maior atenção ao design da hierarquia de classes
 - Para evitar problemas como o fatiamento de objetos e surgimento de ambiguidades, aumentando a complexidade do programa
- Necessidade de se implementar destrutores virtuais
 - Evita o surgimento de comportamento indefinido

Especificador override

- Saída de **exibeValor()** está errada
 - Chamou a função da Base
 - Função com o mesmo nome, mas assinatura diferente
 - Versão da Derivada não substitui a da Base
 - Mesmo que sejam marcadas como virtuais

```
class Base {
public:
    virtual void exibeNome() { cout << "Base\n"; }
    virtual void exibeValor(int x) {
        cout << "Base: " << x << "\n";
    }
};

class Derivada : public Base {
public:
    virtual void exibeNome() { cout << "Derivada\n"; }
    virtual void exibeValor(float x) {
        cout << "Derivada: " << x << "\n";
    }
};

int main() {
    Derivada d;
    Base &b = d;

    b.exibeNome();
    b.exibeValor(5.25);

    return 0;
}
```

Saída:

Derivada
Base: 5

Especificador override

- Problema comum com função virtual
 - As funções devem seguir exatamente o mesmo protótipo
 - Compilador não dá erro
 - Pode levar a bugs difíceis de encontrar

```
class Base {
public:
    virtual void exibeNome() { cout << "Base\n"; }
    virtual void exibeValor(int x) {
        cout << "Base: " << x << "\n";
    }
};

class Derivada : public Base {
public:
    virtual void exibeNome() { cout << "Derivada\n"; }
    virtual void exibeValor(float x) {
        cout << "Derivada: " << x << "\n";
    }
};

int main() {
    Derivada d;
    Base &b = d;

    b.exibeNome();
    b.exibeValor(5.25);

    return 0;
}
```

Saída:

Derivada
Base: 5

Especificador override

- Solução: usar **override** ao sobrescrever métodos (C++11)
 - Colocado ao final da declaração da função na Derivada
 - Não faz uso do virtual
 - Compilador verifica se a função está sobrescrevendo a versão da base
 - Mesmo nome, parâmetros e tipo de retorno compatível

```
class Base {
public:
    virtual void exibeNome() { cout << "Base\n"; }
    virtual void exibeValor(int x) {
        cout << "Base: " << x << "\n";
    }
};

class Derivada : public Base {
public:
    void exibeNome() override { cout << "Derivada\n"; }
    void exibeValor(float x) override { //Erro de compilação
        cout << "Derivada: " << x << "\n";
    }
};

int main(){
    Derivada d;
    Base &b = d;

    b.exibeNome();
    b.exibeValor(5.25);

    return 0;
}
```

Especificador override

- Atua como uma ferramenta de verificação de consistência
 - Ajuda a evitar bugs sutis em tempo de compilação
 - Torna o uso de funções virtuais mais seguro e confiável
- Recomendação de uso
 - **virtual** na classe base
 - **override** na classe derivada

```
class Base {
public:
    virtual void exibeNome() { cout << "Base\n"; }
    virtual void exibeValor(int x) {
        cout << "Base: " << x << "\n";
    }
};

class Derivada : public Base {
public:
    void exibeNome() override { cout << "Derivada\n"; }
    void exibeValor(float x) override { //Erro de compilação
        cout << "Derivada: " << x << "\n";
    }
};

int main(){
    Derivada d;
    Base &b = d;

    b.exibeNome();
    b.exibeValor(5.25);

    return 0;
}
```

Especificador final

- Restringe a sobrescrita de métodos virtuais (C++11)
 - Indica ao compilador que aquela função não pode ser sobrescrita em classes derivadas
 - Nenhuma classe descendente poderá fornecer uma nova implementação
 - Evita extensões ou modificações em determinados pontos da hierarquia
 - **exibeNome()** não pode ser sobrescrita

```
class Ponto{
public:
    virtual void exibeNome() {
        cout << "Ponto\n";
    }
};

class Reta: public Ponto{
public:
    void exibeNome() override final {
        cout << "Reta\n";
    }
};

class Quadrado: public Reta{
public:
    //Erro de compilação
    void exibeNome() override {
        cout << "Quadrado\n";
    }
};
```

Especificador final

- Também pode ser usado para restringir a herança de classes
 - Indica que aquela classe não pode ser herdada
 - A classe Reta não pode ter classes descendentes

```
class Ponto{
public:
    void exibeNome() {
        cout << "Ponto\n";
    }
};

class Reta final: public Ponto{
public:
    void exibeNome() {
        cout << "Reta\n";
    }
};

//Erro de compilação
class Quadrado: public Reta{
public:
    void exibeNome() {
        cout << "Quadrado\n";
    }
};
```

Destrutor Virtual

- Destrutor
 - Método especial que é executado automaticamente quando um objeto de uma classe é removido da memória
 - Libera recursos adquiridos durante a vida útil do objeto
 - Memória alocada dinamicamente
 - Conexões com arquivos ou bancos de dados
 - Ou outros recursos de sistema

Destruitor Virtual

- Importante quando excluimos um objeto derivado por meio de um ponteiro para a classe base
 - Garante que todos os recursos associados ao objeto sejam liberados corretamente
 - Chama os destrutores de todas as classes na ordem adequada

Destrutor Virtual

- Apenas Derivada implementa construtor e destrutor
 - Gerencia a alocação e liberação de memória
- Programa principal
 - Alocação de um novo objeto Derivada em um ponteiro para Base
 - Não houve a chamada para o destrutor da classe
 - Destrutor não é virtual, chama o da Base (inexistente)

```
class Base{
public:
    virtual void exibeNome() {
        cout << "Base\n";
    }
};

class Derivada: public Base{
private:
    int* vetor;
public:
    Derivada(int tamanho){
        cout << "Construtor Derivada\n";
        vetor = new int[tamanho];
    }
    ~Derivada() {
        cout << "Destrutor Derivada\n";
        delete[] vetor;
    }
    void exibeNome() override {
        cout << "Derivada\n";
    }
};

int main(){
    Base *b = new Derivada(10);
    b->exibeNome();
    delete b;

    return 0;
}
```

Saída:

Construtor Derivada
Derivada

Destrutor Virtual

- Se a classe for usada polimorficamente, devemos declarar o destrutor da base como virtual
 - Virtual no destrutor da Base
 - Override no destrutor da Derivada
 - Garante a chamada de todos os destrutores

```
class Base{
public:
    virtual ~Base() { cout << "Destrutor Base\n"; }

    virtual void exibeNome() { cout << "Base\n"; }
};

class Derivada: public Base{
private:
    int* vetor;
public:
    Derivada(int tamanho){
        cout << "Construtor Derivada\n";
        vetor = new int[tamanho];
    }
    ~Derivada() override {
        cout << "Destrutor Derivada\n";
        delete[] vetor;
    }
    void exibeNome() override {
        cout << "Derivada\n";
    }
};

int main(){
    Base *b = new Derivada(10);
    b->exibeNome();
    delete b;

    return 0;
}
```

Saída:

Construtor Derivada
Derivada
Destrutor Derivada
Destrutor Base

Destrutor Virtual

- Solução anterior tem um inconveniente
 - Necessidade de criar um destrutor virtual vazio
- Solução
 - Definir um destrutor virtual **default**
 - Não é necessário implementá-lo
 - Serve apenas para garantir a chamada correta dos destrutores

```
class Base{
public:
    virtual ~Base() = default;

    virtual void exibeNome() { cout << "Base\n"; }
};

class Derivada: public Base{
private:
    int* vetor;
public:
    Derivada(int tamanho){
        cout << "Construtor Derivada\n";
        vetor = new int[tamanho];
    }
    ~Derivada() override {
        cout << "Destrutor Derivada\n";
        delete[] vetor;
    }
    void exibeNome() override {
        cout << "Derivada\n";
    }
};

int main(){
    Base *b = new Derivada(10);
    b->exibeNome();
    delete b;

    return 0;
}
```

Fatiamento de objeto

- *Object slicing*
 - Perda da “parte derivada” de um objeto
 - Mantém apenas o subobjeto da Base
 - Como copiar um texto completo para uma folha menor
 - Só a parte que “cabe” (a base) é mantida
 - Tudo o que pertence à derivada é descartado
 - Similar a guardar um **float** em um **int**

```
class Base {
public:
    int x;
    Base(int v) : x(v) {}
    virtual void mostrar() const {
        cout << "Base: x = " << x << endl;
    }
};

class Derivada : public Base {
public:
    int y;
    Derivada(int v1, int v2) : Base(v1), y(v2) {}
    void mostrar() const override {
        cout << "Derivada: x = " << x << ", y = " << y << endl;
    }
};
```

Fatiamento de objeto

- Ocorre quando ele é passado por valor
 - Objetos polimórficos nunca devem ser passados ou armazenados por valor da base
 - Sempre devemos usar ponteiros/referências
- Ocorre na
 - Atribuição/Inicialização por valor
 - Passagem de parâmetros por valor
 - Retorno de função por valor

```
void funcao_recebe(Base b) {  
    b.mostrar();  
}  
  
Base funcao_retorna() {  
    // Fatiamento do retorno da função  
    Derivada d(3, 300);  
    return d;  
}  
  
int main() {  
    Derivada d(1, 100);  
    // Fatiamento do objeto na atribuição  
    Base b1 = d;  
    // Fatiamento na passagem de parâmetro  
    funcao_recebe(d);  
    // Fatiamento na construção de cópia  
    Base b2(d);  
  
    return 0;  
}
```

Função e Classe Abstrata

- Função abstrata ou virtual pura
 - Apenas define o protótipo sem implementar
 - Não tem implementação na classe base
 - É marcada com o valor = 0
 - Precisa obrigatoriamente ser sobrescrita em pela classe derivada

```
class Pessoa { // Classe Abstrata
protected:
    string nome;
public:
    Pessoa(string _nome): nome(_nome){}
    void exibirNome() {
        cout << "Nome: " << nome << endl;
    }
    // Função Abstrata ou Virtual Pura
    virtual void exibirCargo() = 0;
};

class Empregado : public Pessoa {
private:
    string cargo;
public:
    Empregado(string _nome, string _cargo)
        : Pessoa(_nome), cargo(_cargo) { }

    void exibirCargo() override {
        cout << "Cargo: " << cargo << endl;
    }
};

int main() {
    //Erro de compilação
    //Pessoa pe("Carlos");

    Empregado em("Eduardo", "Professor");
    em.exibirNome();
    em.exibirCargo();

    return 0;
}
```

Função e Classe Abstrata

- Classe abstrata
 - Uma classe que contenha pelo menos uma função abstrata
 - Não pode ser instanciada diretamente
 - Serve apenas como modelo para outras classes

```
class Pessoa { // Classe Abstrata
protected:
    string nome;
public:
    Pessoa(string _nome): nome(_nome){}
    void exibirNome() {
        cout << "Nome: " << nome << endl;
    }
    // Função Abstrata ou Virtual Pura
    virtual void exibirCargo() = 0;
};
```

```
class Empregado : public Pessoa {
private:
    string cargo;
public:
    Empregado(string _nome, string _cargo)
        : Pessoa(_nome), cargo(_cargo) { }

    void exibirCargo() override {
        cout << "Cargo: " << cargo << endl;
    }
};
```

```
int main() {
    //Erro de compilação
    //Pessoa pe("Carlos");

    Empregado em("Eduardo", "Professor");
    em.exibirNome();
    em.exibirCargo();

    return 0;
}
```


Função e Classe Abstrata

- Podemos definir que a função é abstrata e ainda assim fornecer a implementação
 - Implementação deve ser feita fora da classe
 - A classe base continua sendo abstrata
 - Não pode ser instanciada
 - A classe derivada deve implementar essa função
 - Mas agora também pode fazer uso da versão fornecida pela base

```
class Pessoa { // Classe Abstrata
protected:
    string nome;
public:
    Pessoa(string _nome): nome(_nome){}

    void exibirNome() {
        cout << "Nome: " << nome << endl;
    }
    // Função Abstrata ou Virtual Pura
    virtual void exibirCargo() = 0;
};

// Implementação da Função Abstrata
void Pessoa::exibirCargo() {
    cout << "Cargo: ??????" << endl;
}

class Empregado : public Pessoa {
private:
    string cargo;
public:
    Empregado(string _nome, string _cargo)
        : Pessoa(_nome), cargo(_cargo) { }

    void exibirCargo() override {
        // Usa a implementação da classe base
        Pessoa::exibirCargo();
    }
};
```

Interface de classe

- É uma classe especial projetada para definir apenas contratos que outras classes devem cumprir
 - Não fornecer nenhuma implementação concreta
 - É uma classe abstrata pura
 - Contém apenas funções abstratas e nenhum atributo ou implementação real
 - Não permite criar objetos diretamente a partir dela

Interface de classe

- Define uma hierarquia de classes
 - A classe base especifica “o que deve ser feito” (assinatura das funções)
 - As classes derivadas decidem “como fazer” (implementação concreta)
- Permite que o programador crie diferentes implementações que compartilham a mesma estrutura de uso
 - Garante consistência
 - Promove baixo acoplamento
 - Alta flexibilidade no código
 - Facilitar o uso de polimorfismo dinâmico

Interface de classe

- Permite modelar sistemas complexos, como

- Bibliotecas gráficas
- Sistemas de arquivos
- Drivers de dispositivos

- Em que diferentes implementações podem compartilhar a mesma interface

```
// Interface de classe para Pessoa
class IPessoa {
public:
    // Funções Abstratas ou Virtuais Puras (contrato)
    virtual void setNome(const string& nome) = 0;
    virtual string getNome() const = 0;
    virtual void setIdade(int idade) = 0;
    virtual int getIdade() const = 0;
    virtual void exibir() const = 0;

    // Destrutor virtual
    virtual ~IPessoa() = default;
};
```